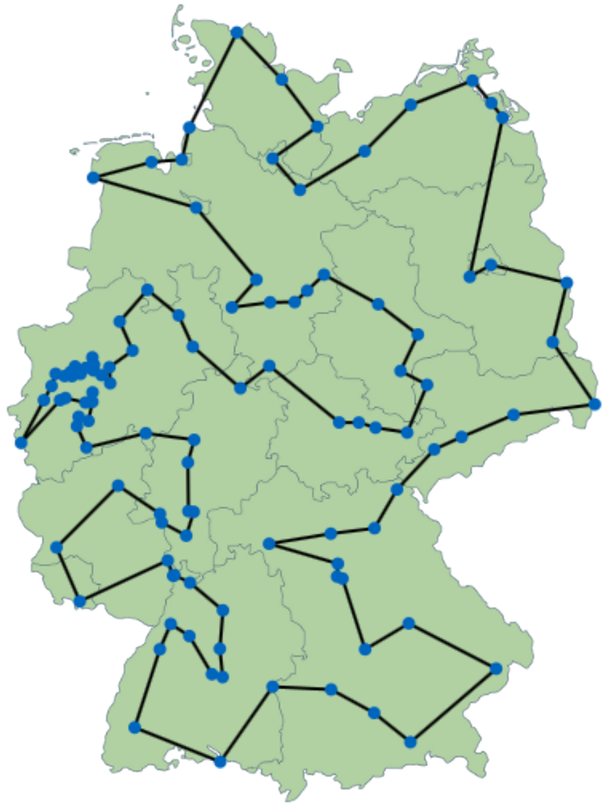
Extension du problème du voyageur pour une optimisation collective

Romain Lo Bello
41701

Définition et modélisation

Problème du voyageur de commerce :



[1]

- Chemin optimal pour relier des villes
- Minimise une variable (temps de trajet, distance, coût,...)

Sommaire

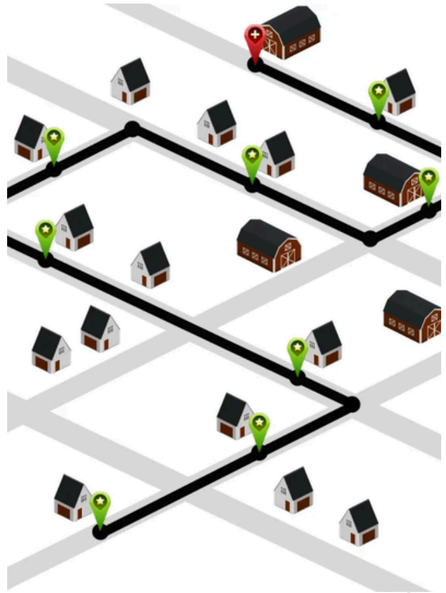
I/ Applications et modélisation du problème

II/ Algorithmes de résolution

III/ Adaptation pour un réseau partagé

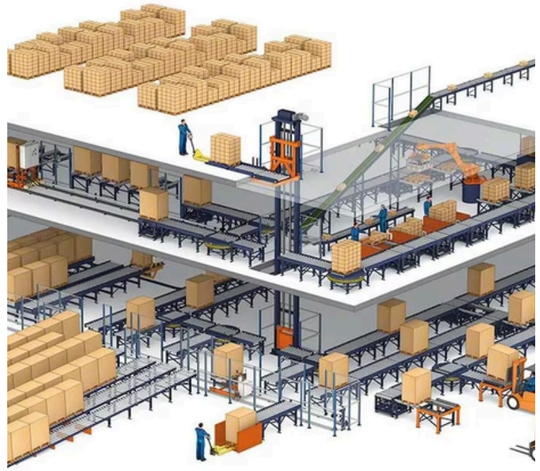
IV/ Résultats et conclusion

I/ Définition et modélisation



[2]

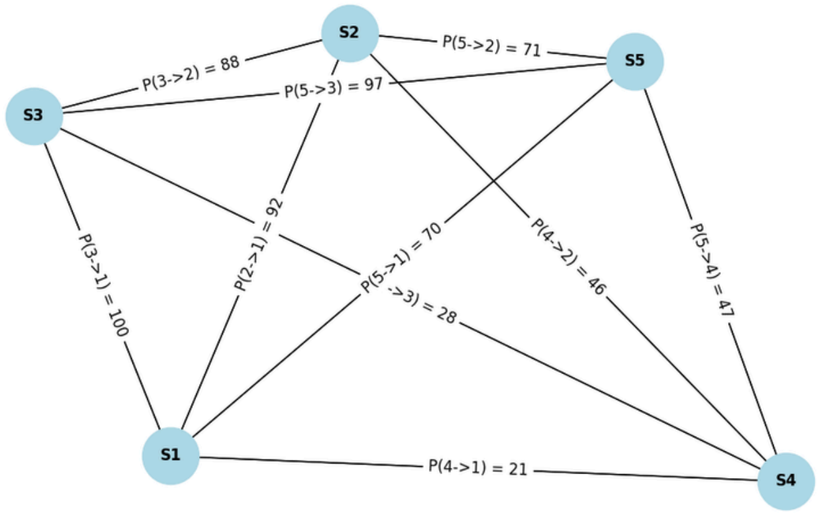
Logistique



[3]

Chaîne de production

I/ Définition et modélisation



$$\begin{bmatrix} 0 & 92 & 100 & 21 & 70 \\ 92 & 0 & 88 & 46 & 71 \\ 100 & 88 & 0 & 28 & 97 \\ 21 & 46 & 28 & 0 & 47 \\ 70 & 71 & 97 & 47 & 0 \end{bmatrix}$$

Graphe Pondéré

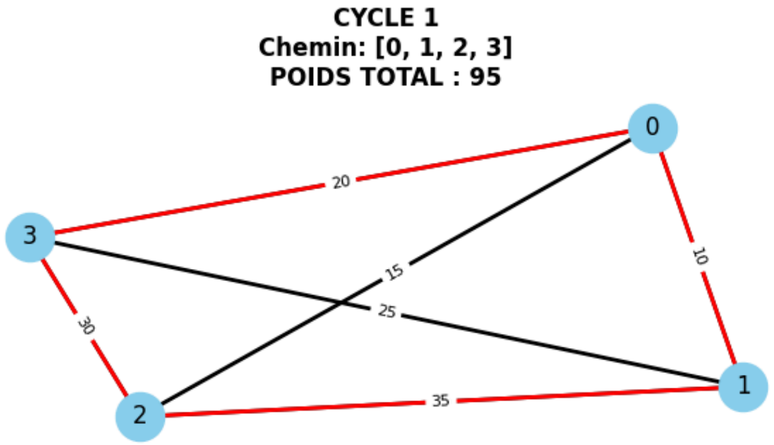
↓

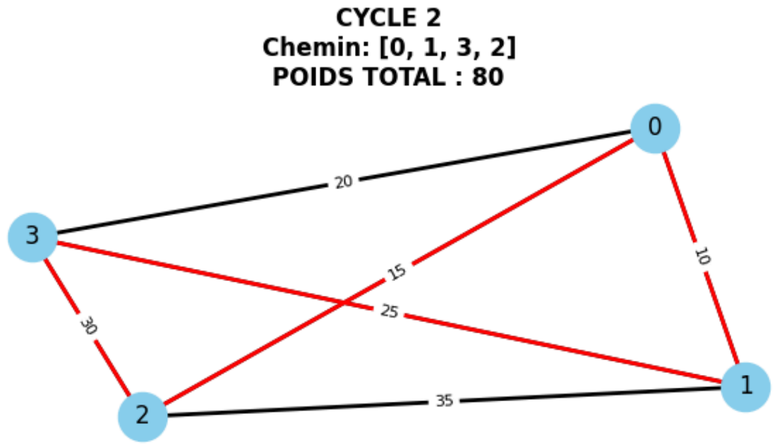
Cycle hamiltonien

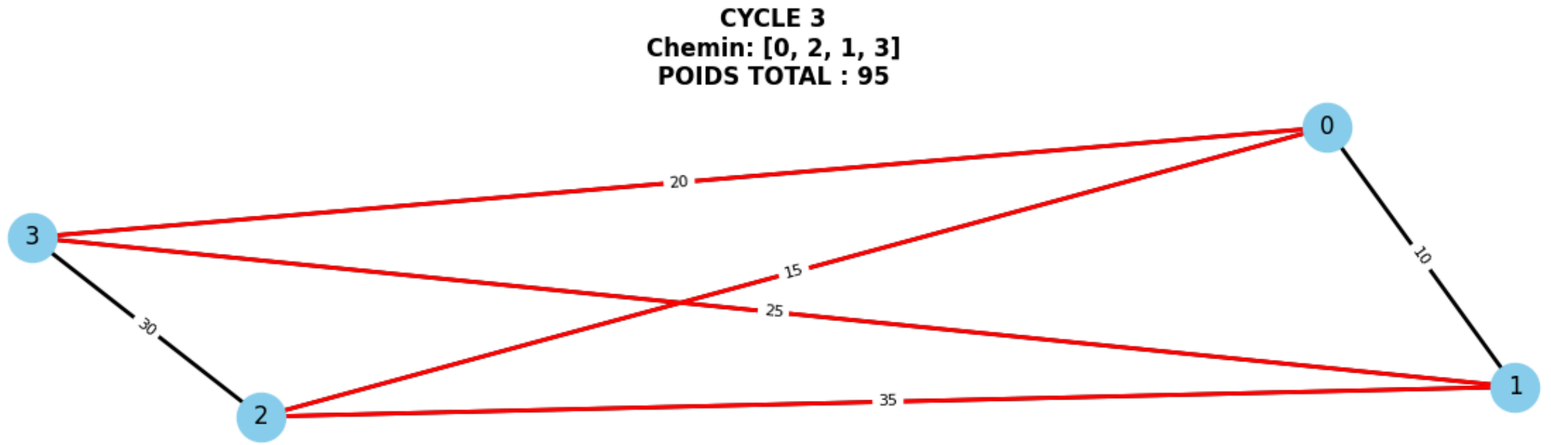
II/ Algorithmes de résolution

Force brute

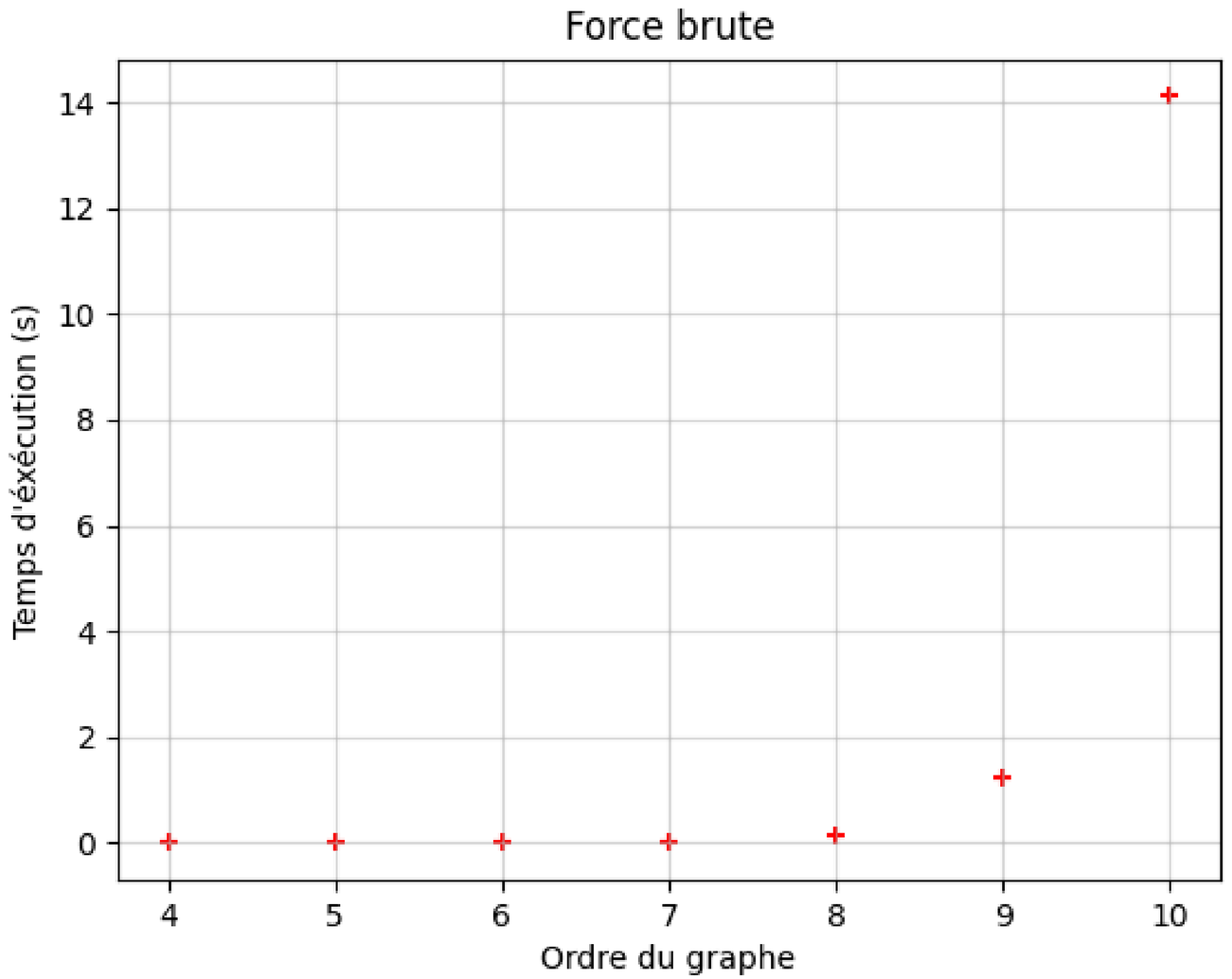
- Essayer tous les cycles possible
- Complexité en $O\left(n \times n!\right)$







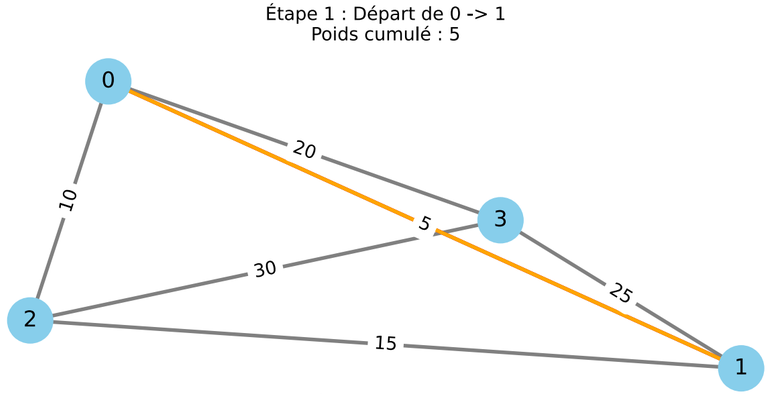
II/ Algorithmes de résolution

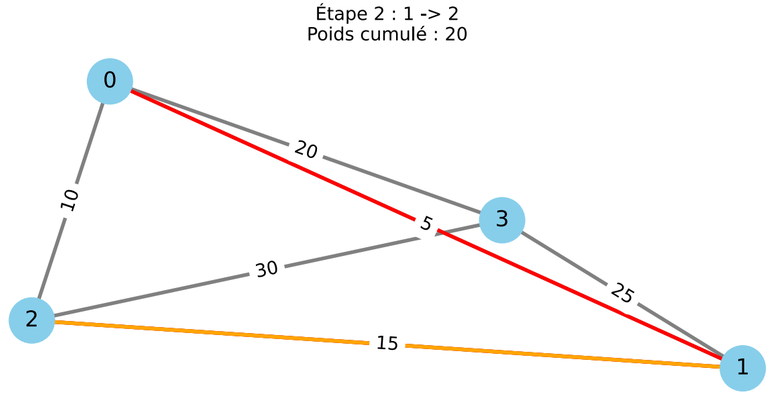


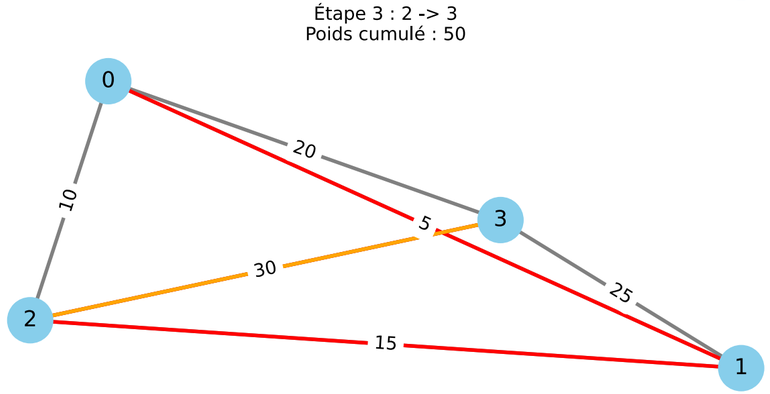
II/ Algorithmes de résolution

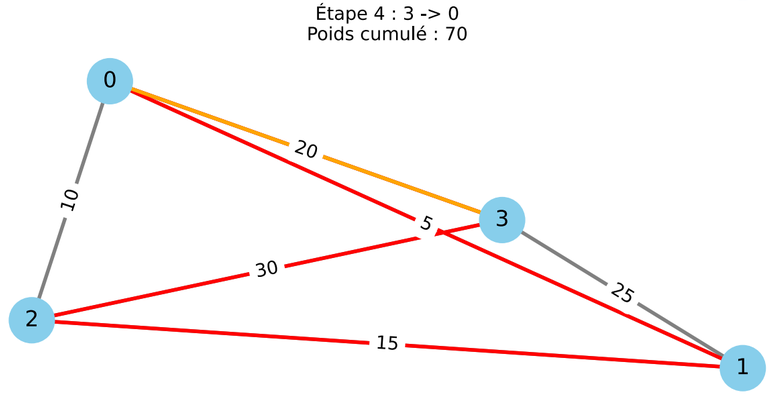
Algorithme glouton

- Optimisation locale
- Approche heuristique
- Complexité en $O\left(n^2\right)$



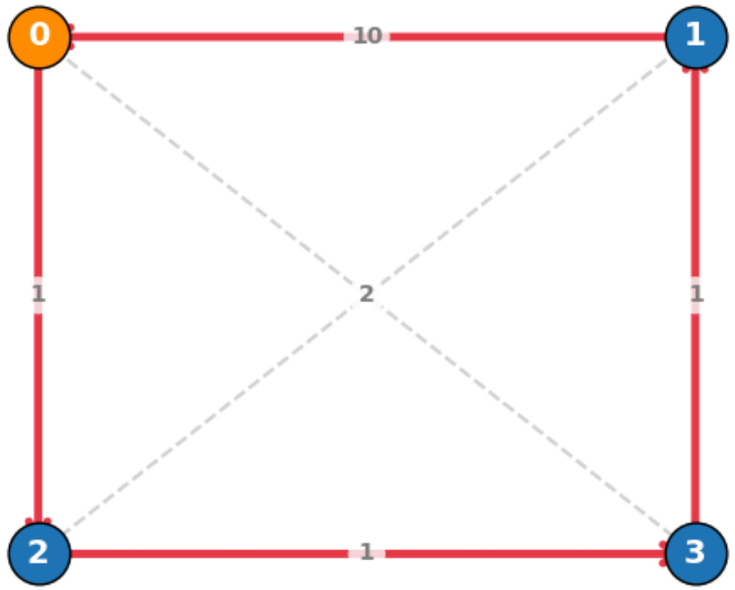




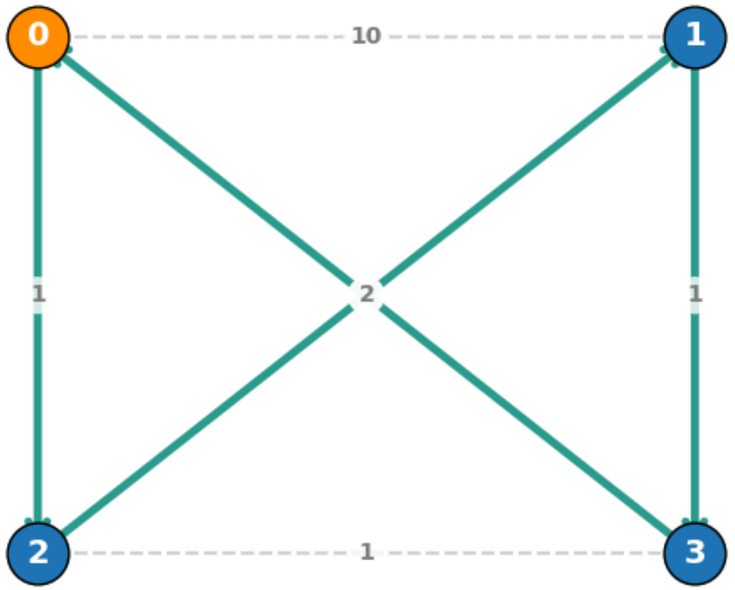


II/ Algorithmes de résolution

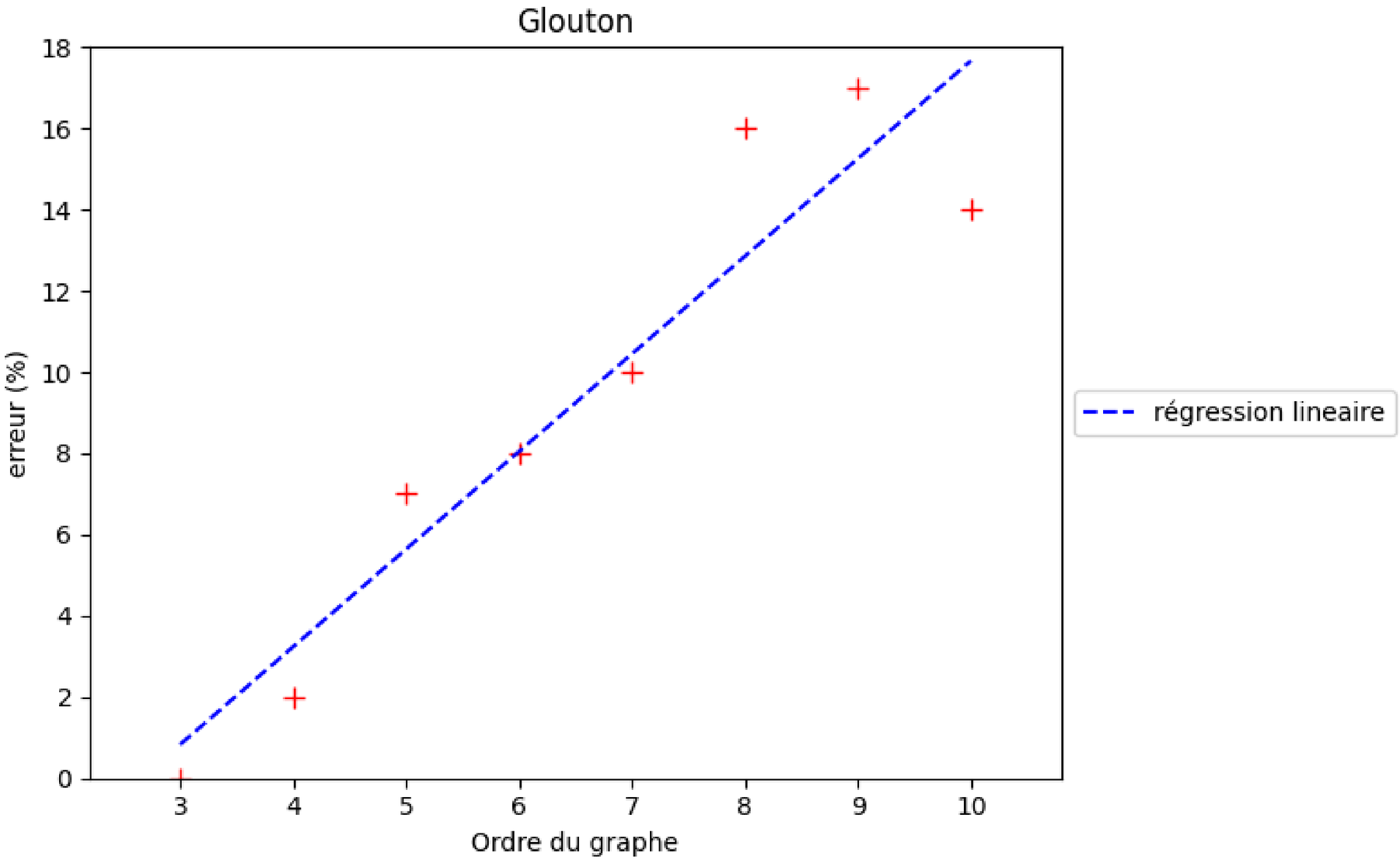
Algorithme Glouton
Cycle : [0, 2, 3, 1, 0]
Coût Total = 13



Force Brute (Exact)
Cycle : [0, 2, 1, 3, 0]
Coût Total = 6



II/ Algorithmes de résolution



II/ Algorithmes de résolution

Algorithme Held-Karp

- Résolution exacte
- Programmation dynamique
- Complexité en $O\left(n^2 \times 2^n\right)$

$G = (S, A); (s_1, s_2) \in S; V \subset S$

$P_{s_1 \to s_2, V}$: Poids du chemin reliant s_1 à s_2 en passant par tous les sommets de V

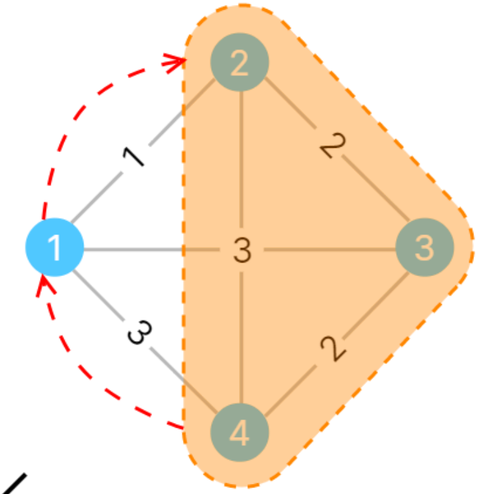
$$P_{s_1 \to s_2, V} = min_{s_i \in V} \left(P_{s_1 \to s_i, V \setminus \{s_i\}} + P_{s_i \to s_2, \{\}} \right)$$

En particulier :

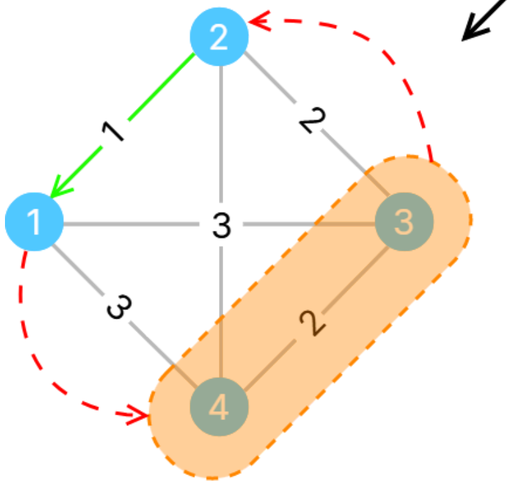
$$P_{s_1 \to s_1, S \setminus \{s_1\}} = min_{s_i \in S \setminus \{s_1\}} \left(P_{s_1 \to s_i, S \setminus \{s_1, s_i\}} + P_{s_i \to s_1, \{\}} \right)$$

II/ Algorithmes de résolution

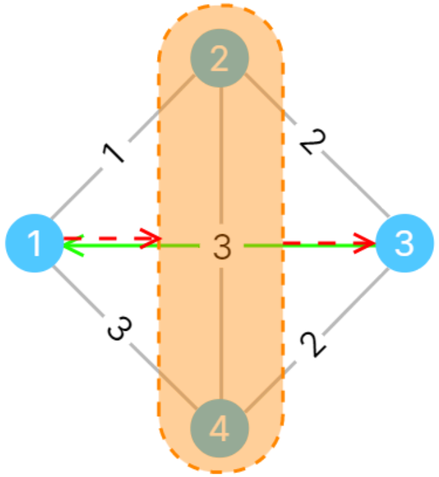
$P_{1\to1,\{2,3,4\}}$



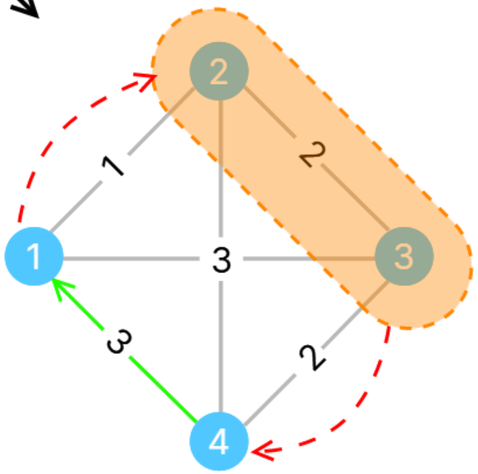
$P_{1\to2,\{3,4\}} + P_{2\to1,\{\}}$



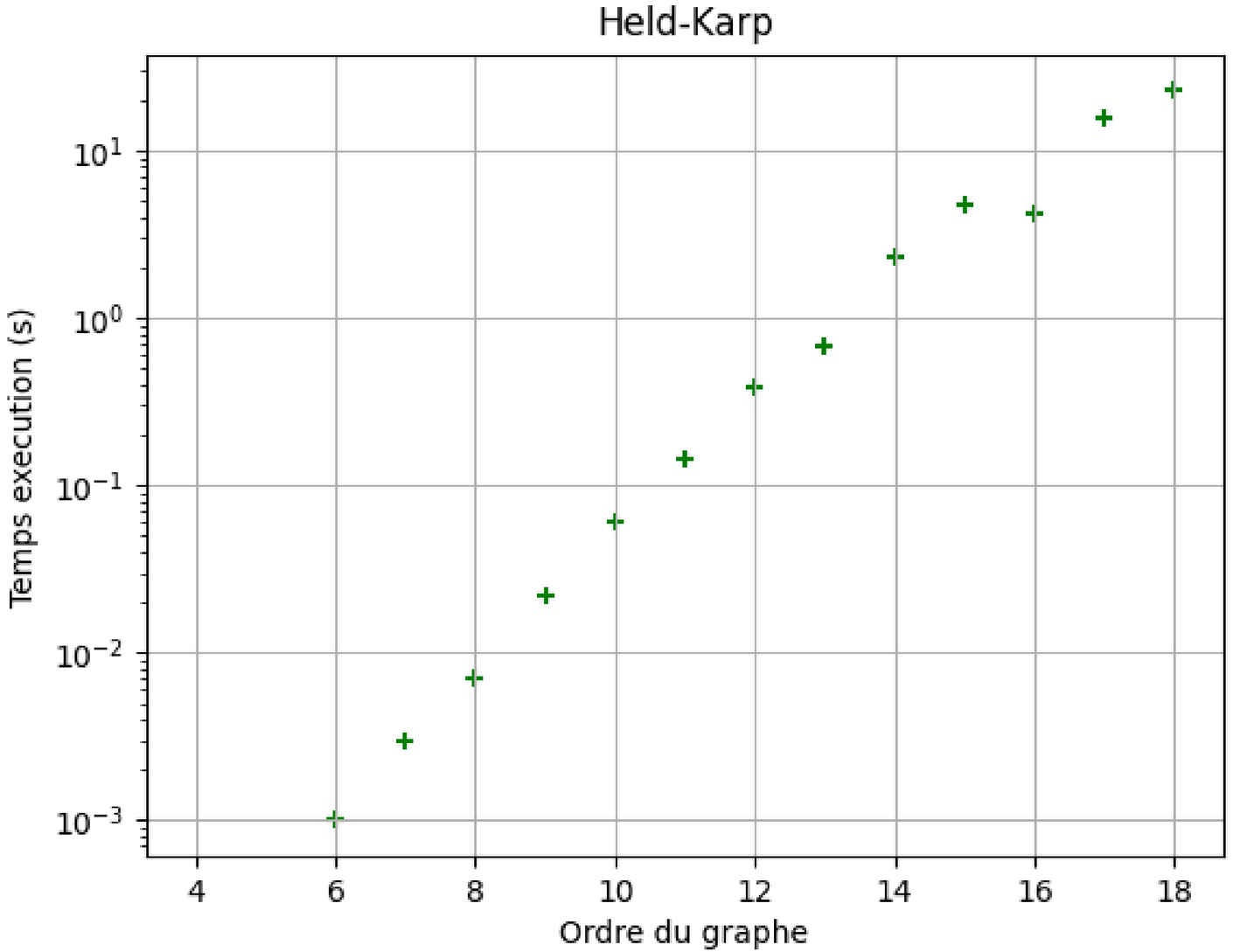
$P_{1\to3,\{2,4\}} + P_{3\to1,\{\}}$



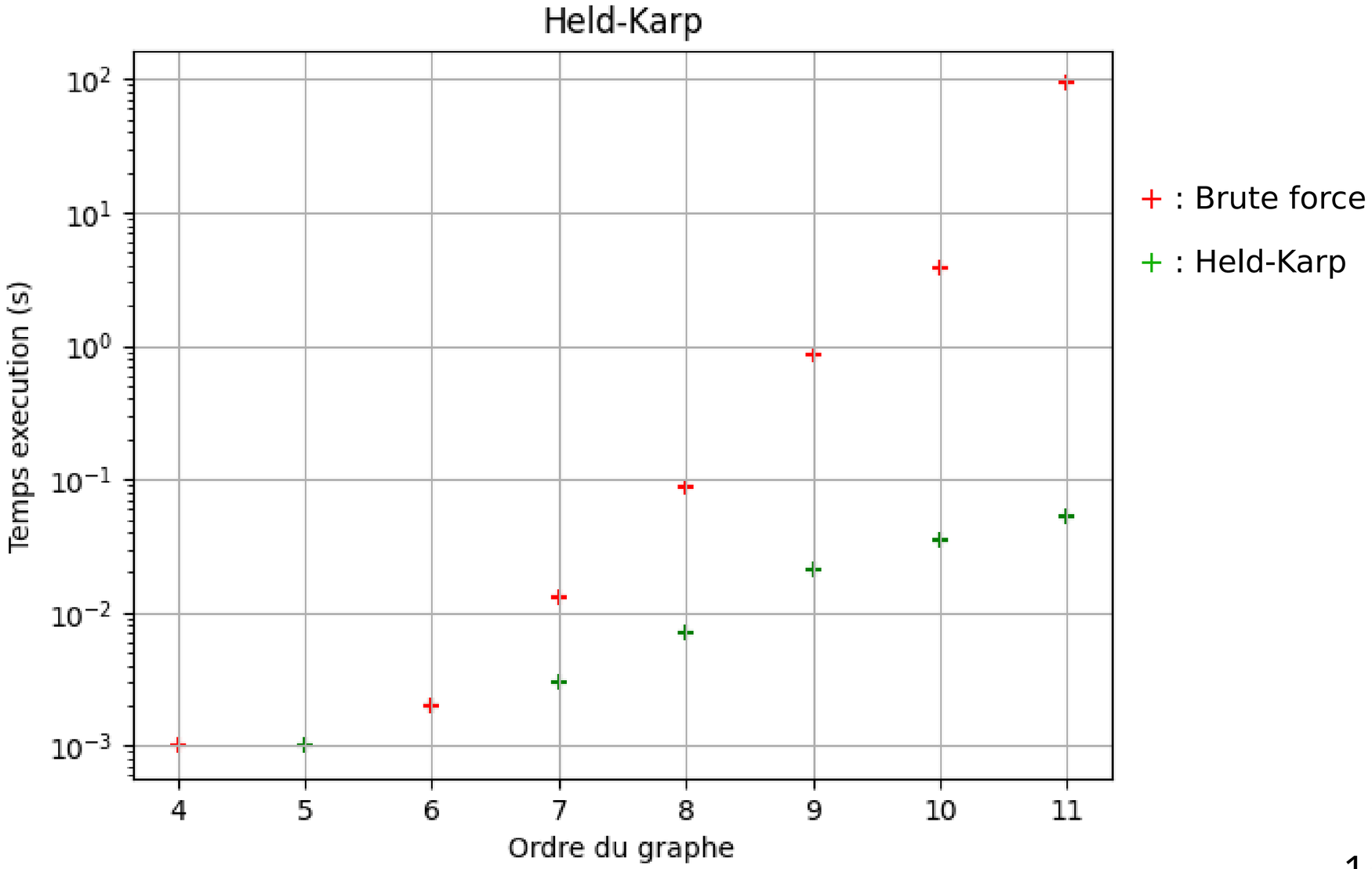
$P_{1\to4,\{2,3\}} + P_{4\to1,\{\}}$



II/ Algorithmes de résolution

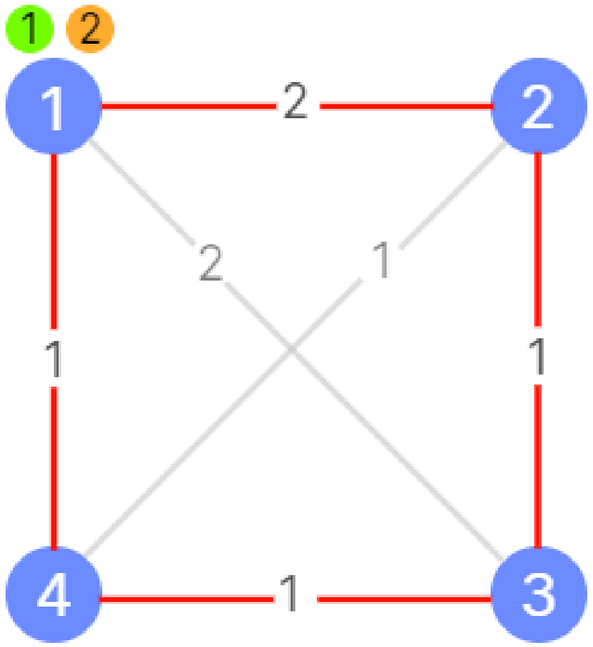


II/ Algorithmes de résolution



III/ Adaptation au problème

- Réseau partagé ⟶ Modification du poids des arrêtes



0

(1,2) [1,2]

(2,3)

(3,4)

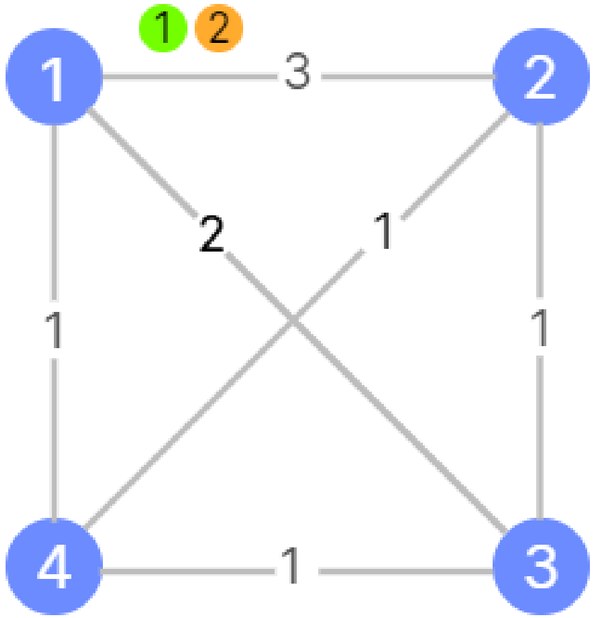
(4,1)

(1,3)

(2,4)

III/ Adaptation au problème

- Réseau partagé ⟶ Modification du poids des arrêtes



1

(1,2) [1,2]

(2,3)

(3,4)

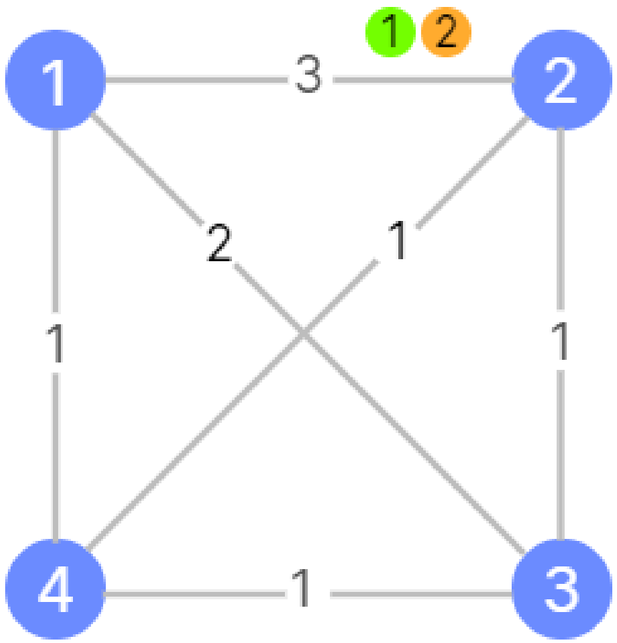
(4,1)

(1,3)

(2,4)

III/ Adaptation au problème

- Réseau partagé ⟶ Modification du poids des arrêtes



2

(1,2) [1,2]

(2,3)

(3,4)

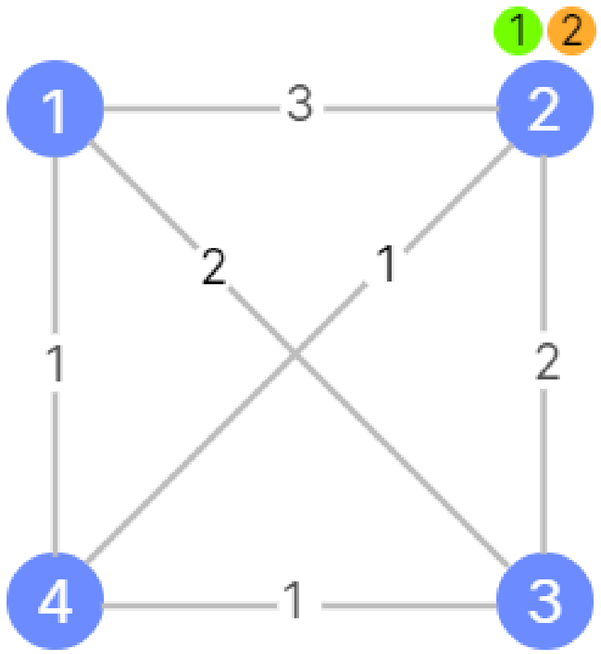
(4,1)

(1,3)

(2,4)

III/ Adaptation au problème

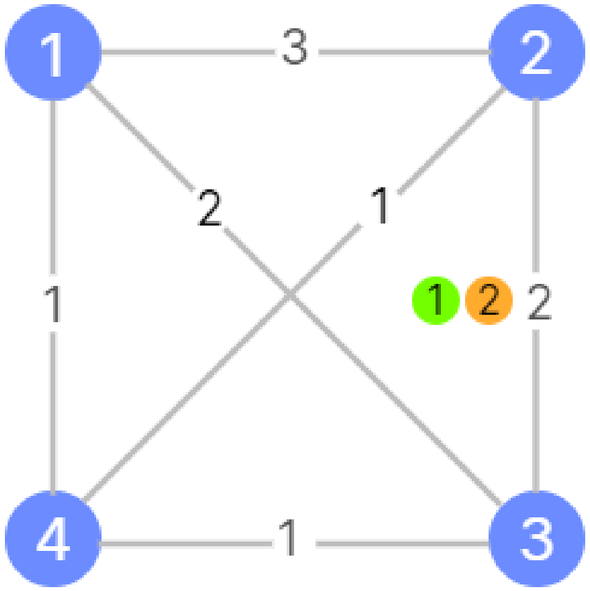
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	3
(1,2)	[1,2]	
(2,3)		[1,2]
(3,4)		
(4,1)		
(1,3)		
(2,4)		

III/ Adaptation au problème

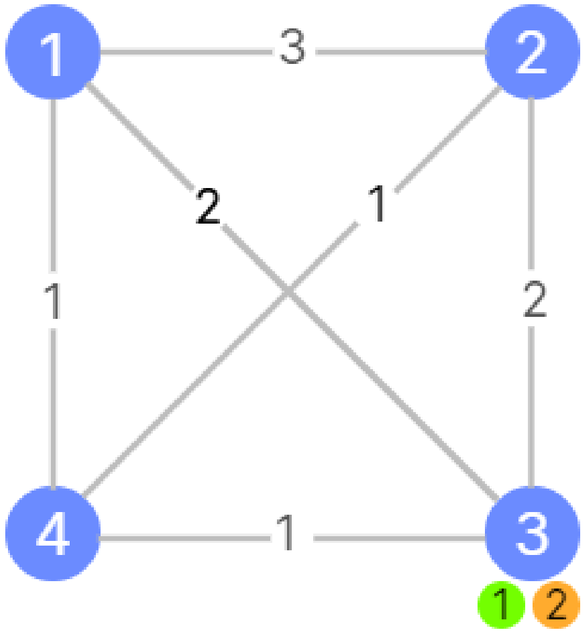
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	4
(1,2)	[1,2]	
(2,3)		[1,2]
(3,4)		
(4,1)		
(1,3)		
(2,4)		

III/ Adaptation au problème

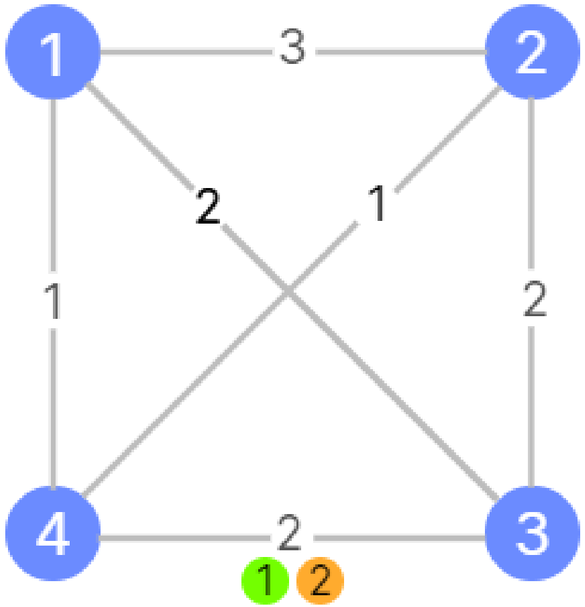
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	3	5
(1,2)	[1,2]		
(2,3)		[1,2]	
(3,4)			[1,2]
(4,1)			
(1,3)			
(2,4)			

III/ Adaptation au problème

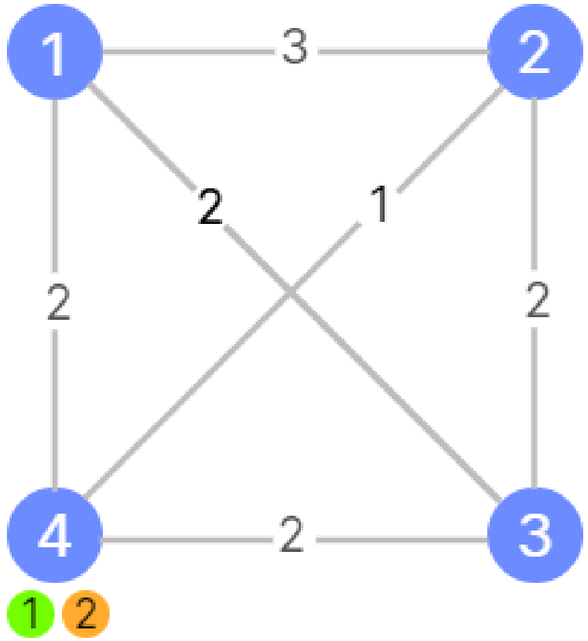
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	3	6
(1,2)	[1,2]		
(2,3)		[1,2]	
(3,4)			[1,2]
(4,1)			
(1,3)			
(2,4)			

III/ Adaptation au problème

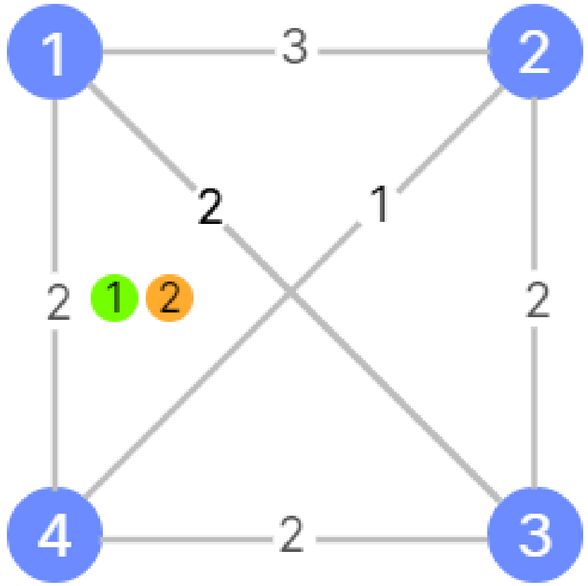
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	3	5	7
(1,2)	[1,2]			
(2,3)		[1,2]		
(3,4)			[1,2]	
(4,1)				[1,2]
(1,3)				
(2,4)				

III/ Adaptation au problème

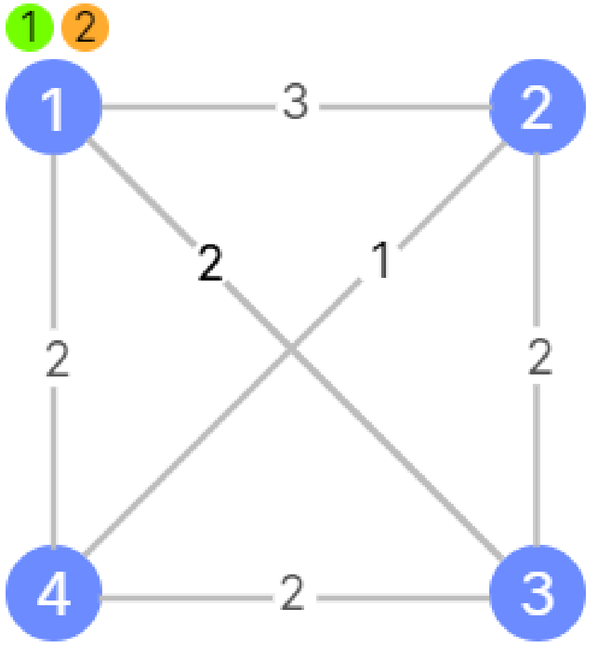
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	3	5	8
(1,2)	[1,2]			
(2,3)		[1,2]		
(3,4)			[1,2]	[1,2]
(4,1)				
(1,3)				
(2,4)				

III/ Adaptation au problème

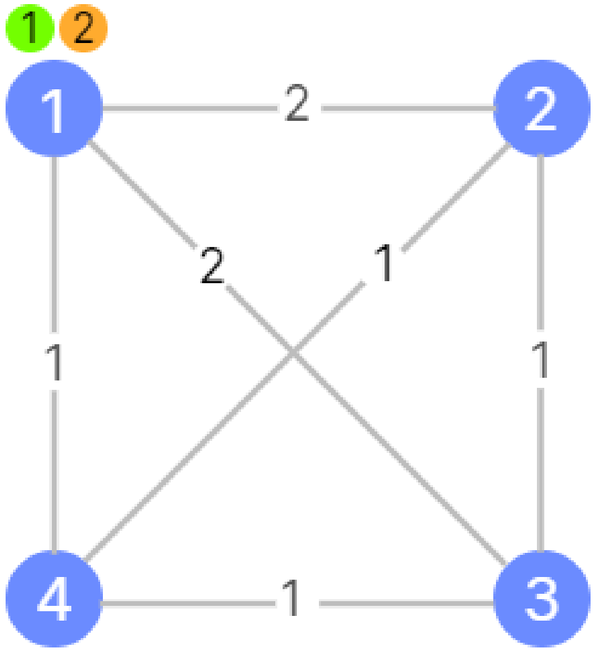
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	3	5	7	9
(1,2)	[1,2]				[1,2]
(2,3)		[1,2]			
(3,4)			[1,2]		
(4,1)				[1,2]	
(1,3)					
(2,4)					

III/ Adaptation au problème

- Réseau partagé ⟶ Modification du poids des arrêtes



0

(1,2) [1,2]

(2,3)

(3,4)

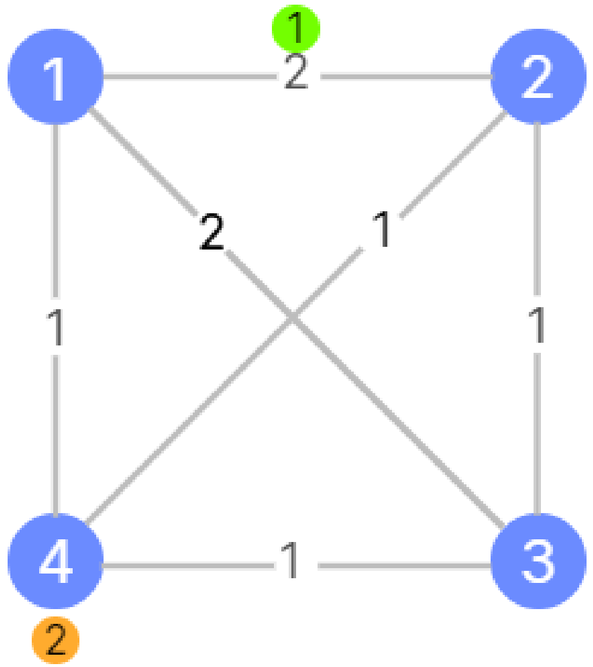
(4,1)

(1,3)

(2,4)

III/ Adaptation au problème

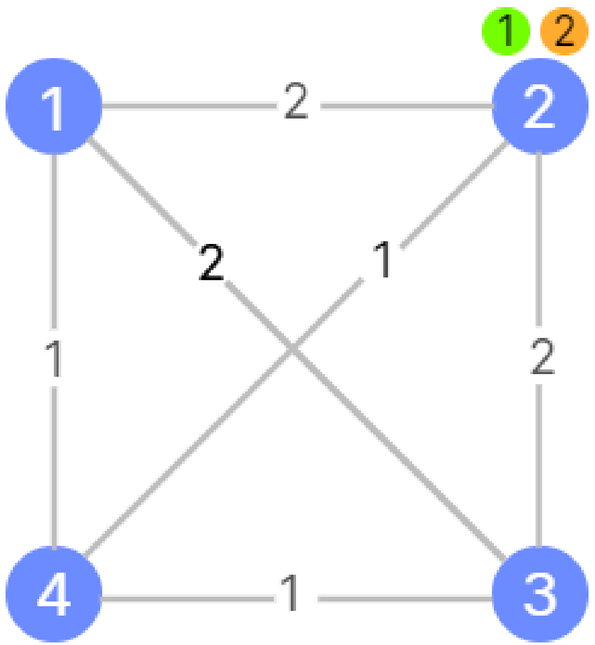
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	1
(1,2)	[1,2]	[1]
(2,3)		
(3,4)		[2]
(4,1)		
(1,3)		
(2,4)		

III/ Adaptation au problème

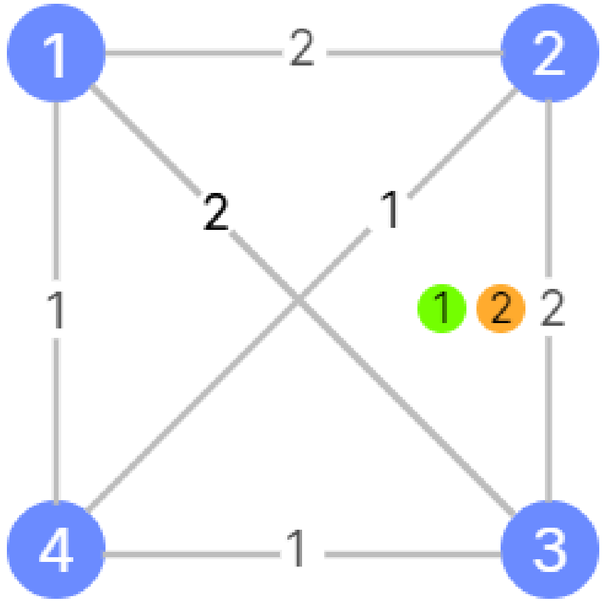
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	1	2
(1,2)	[1,2]	[1]	[1,2]
(2,3)			
(3,4)		[2]	
(4,1)			
(1,3)			
(2,4)			

III/ Adaptation au problème

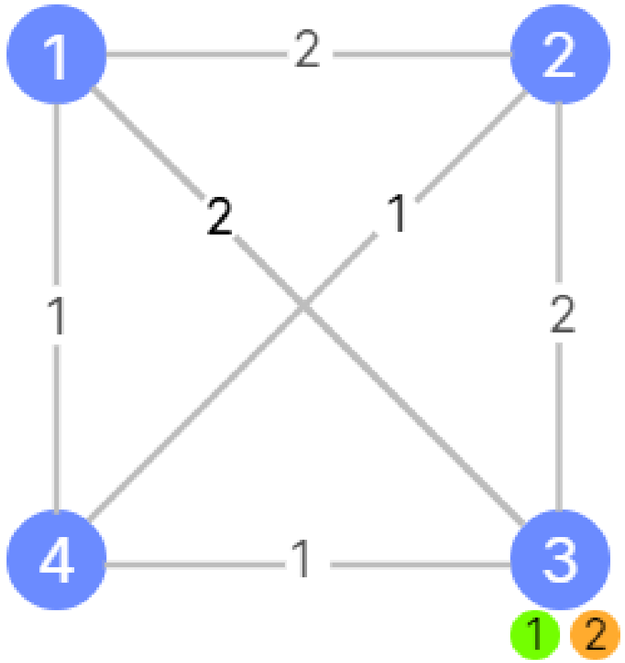
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	1	2	3
(1,2)	[1,2]	[1]	[1,2]	[1,2]
(2,3)				
(3,4)		[2]		
(4,1)				
(1,3)				
(2,4)				

III/ Adaptation au problème

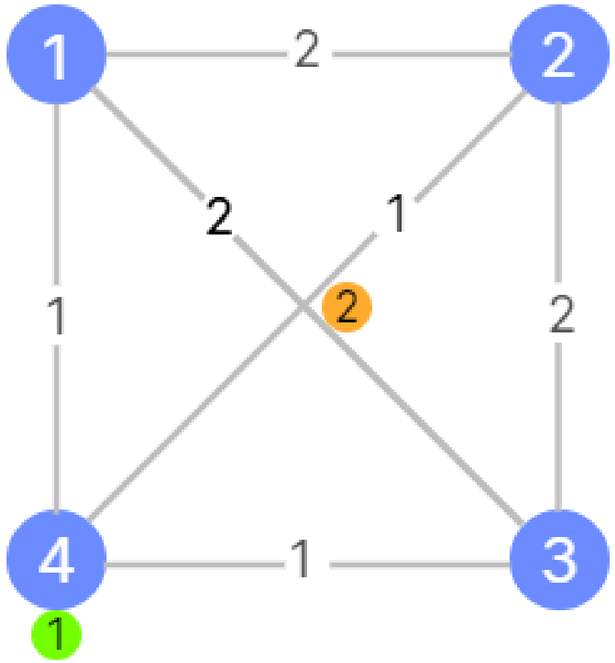
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	1	2	3	4
(1,2)	[1,2]	[1]	[1,2]	[1,2]	
(2,3)					
(3,4)		[2]			[1]
(4,1)					
(1,3)					[2]
(2,4)					

III/ Adaptation au problème

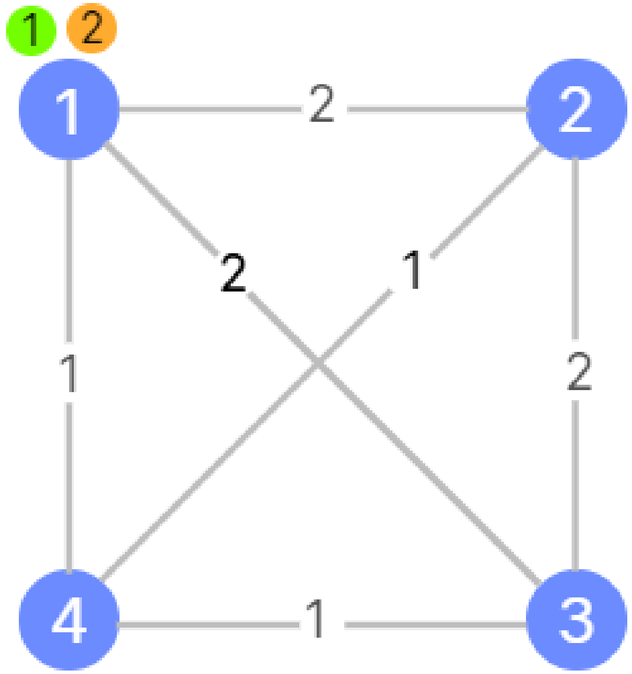
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	1	2	3	4	5
(1,2)	[1,2]	[1]	[1,2]	[1,2]		
(2,3)						
(3,4)		[2]			[1]	
(4,1)						[1]
(1,3)					[2]	[2]
(2,4)						

III/ Adaptation au problème

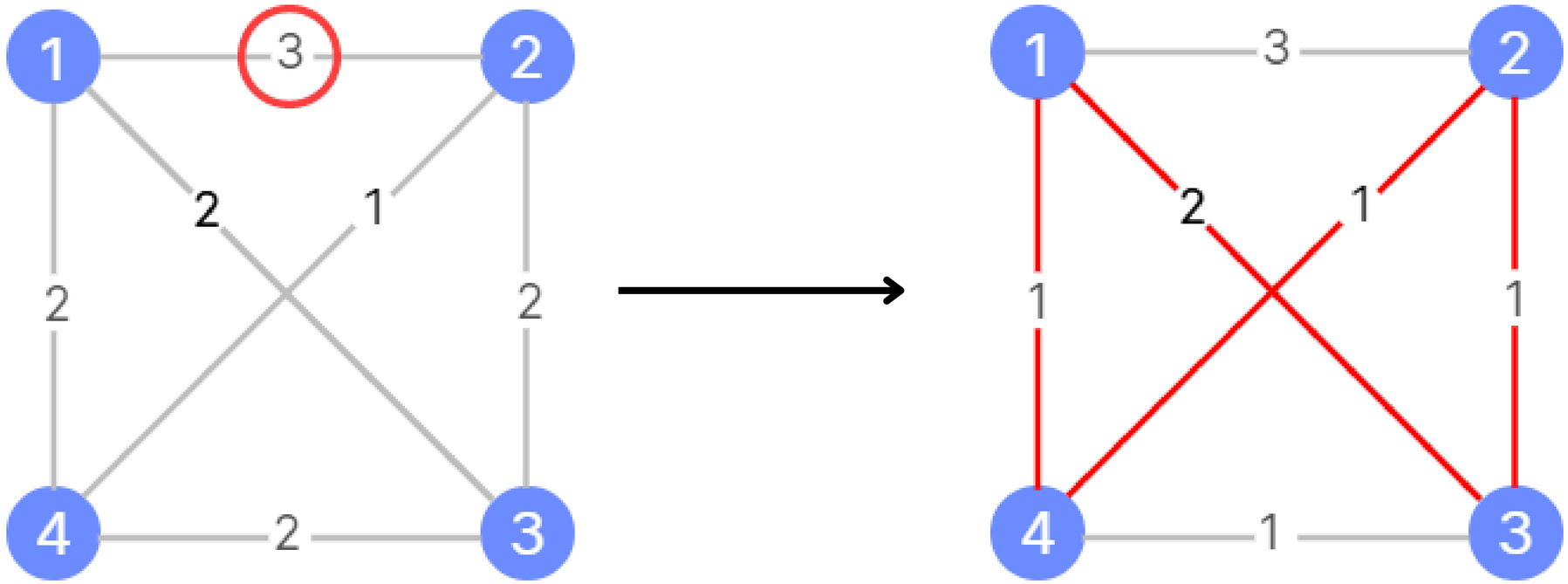
- Réseau partagé ⟶ Modification du poids des arrêtes



	0	1	2	3	4	5	6
(1,2)	[1,2]	[1]	[1,2]	[1,2]			[1,2]
(2,3)							
(3,4)		[2]			[1]		
(4,1)						[1]	
(1,3)					[2]	[2]	
(2,4)							

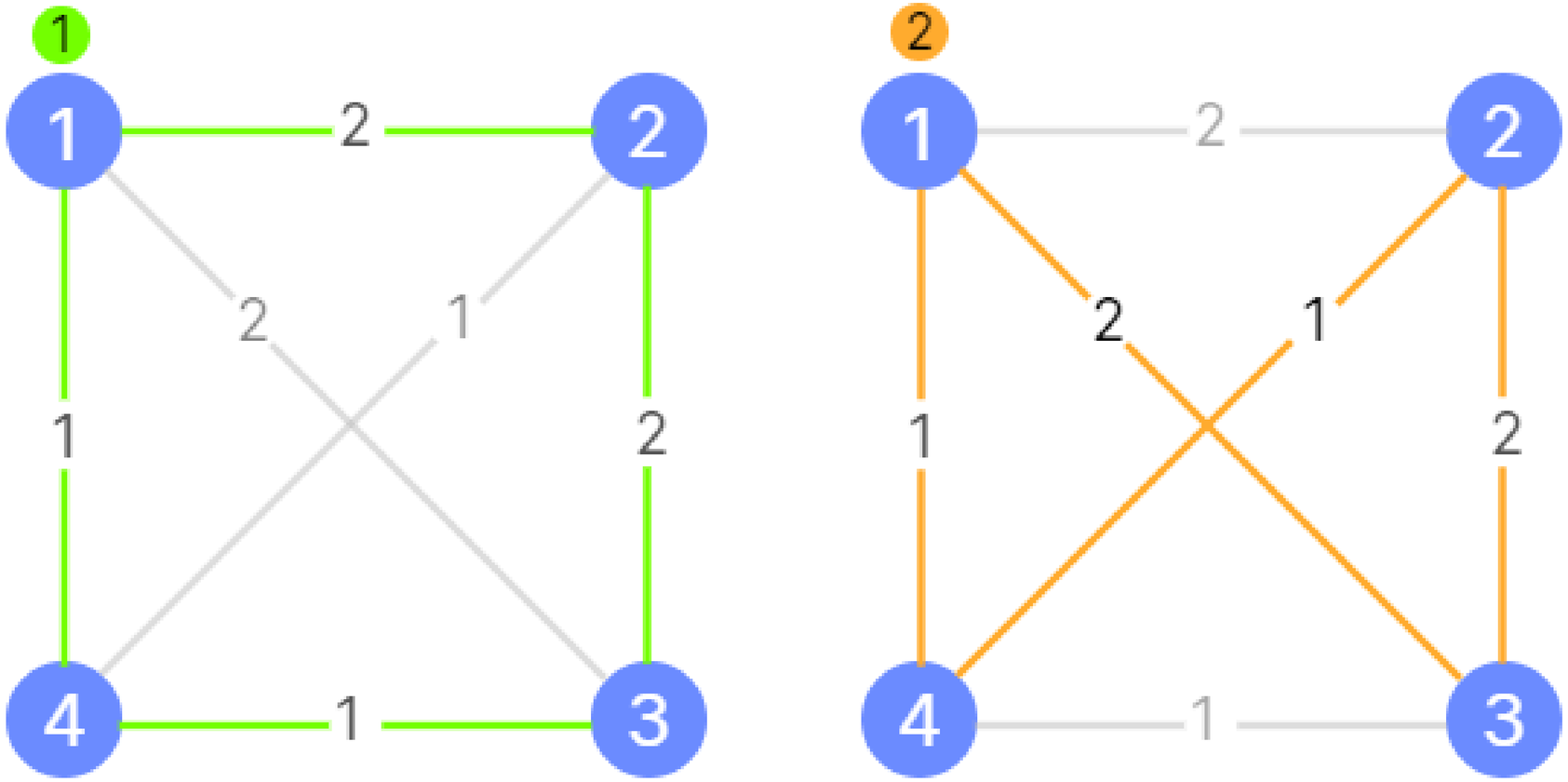
III/ Adaptation au problème

Algorithme de résolution



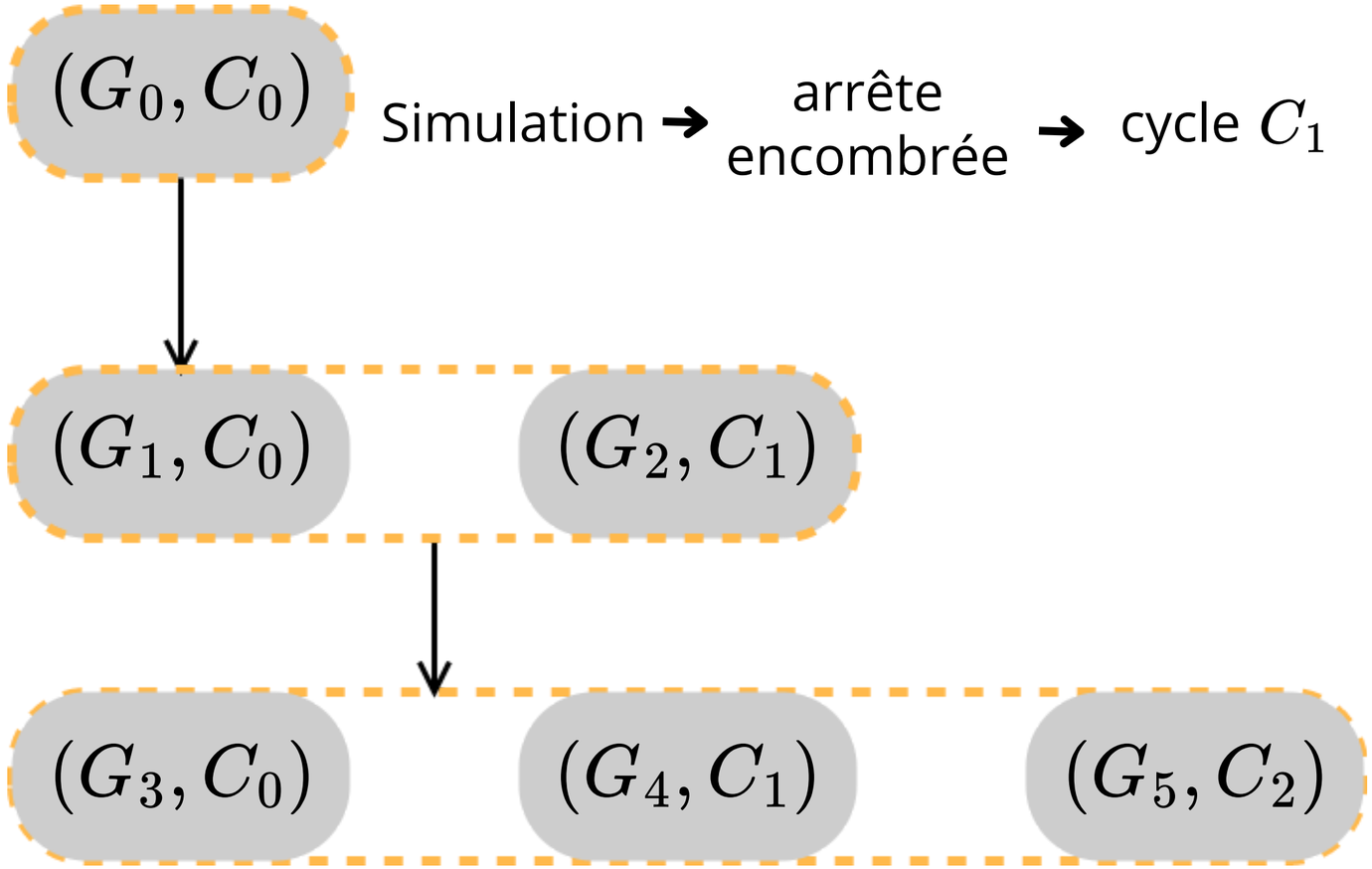
III/ Adaptation au problème

Algorithme de résolution



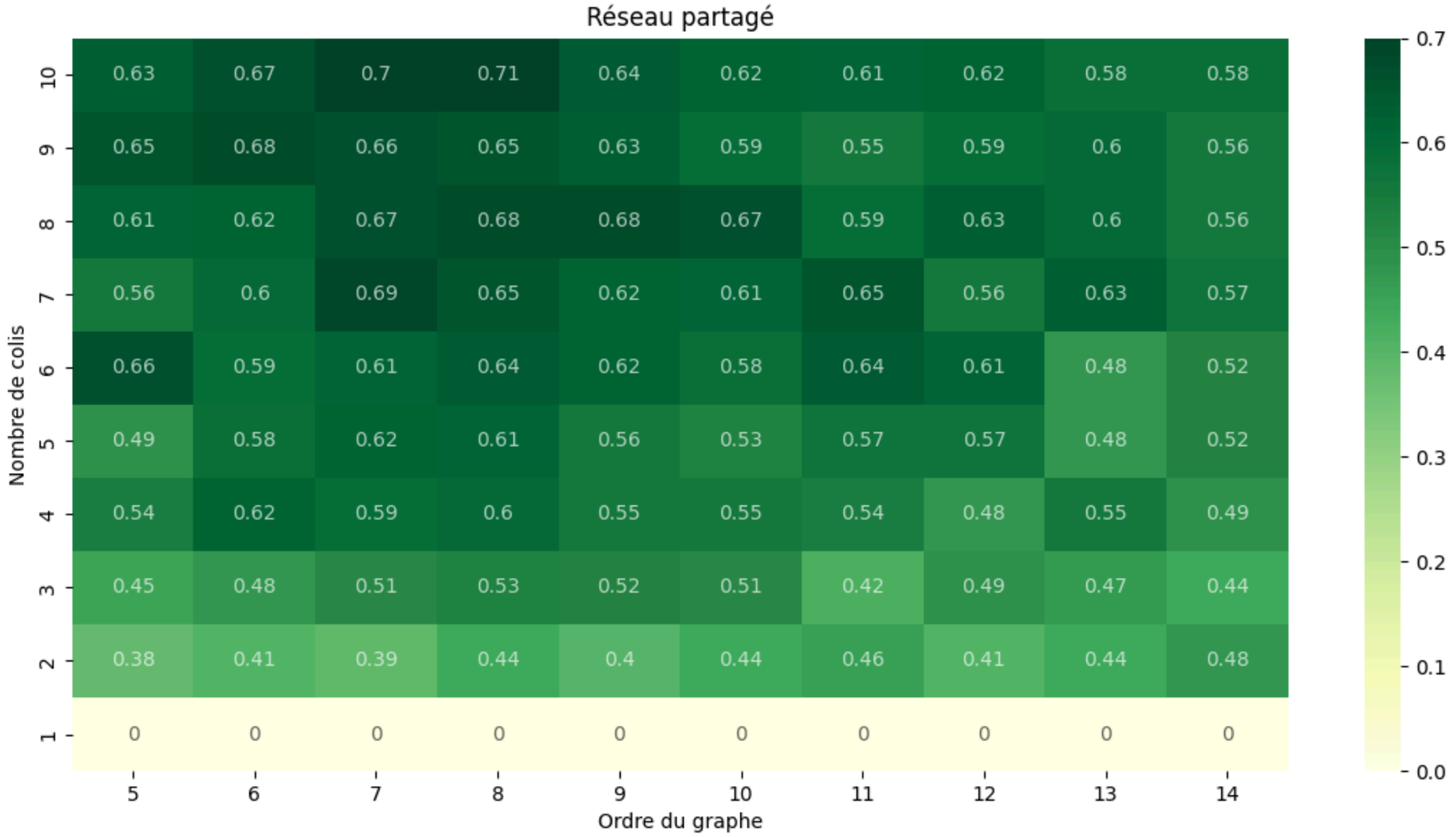
III/ Adaptation au problème

Algorithme de résolution

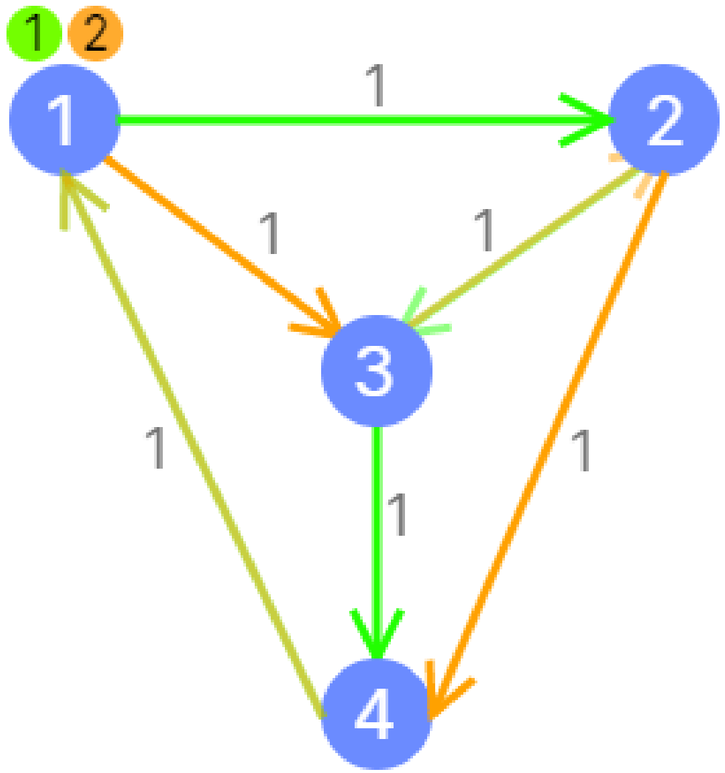


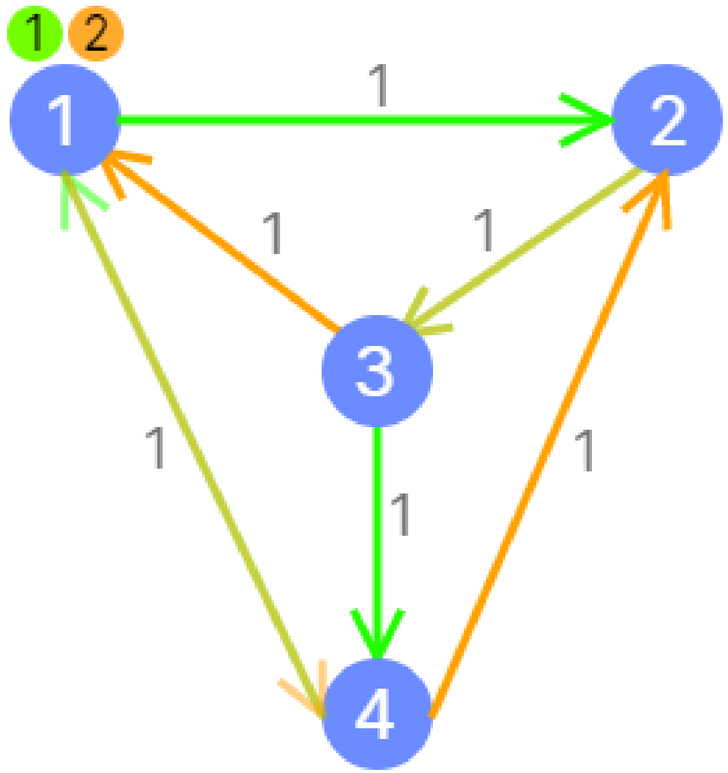
IV/ Résultats et conclusion

Algorithme de résolution

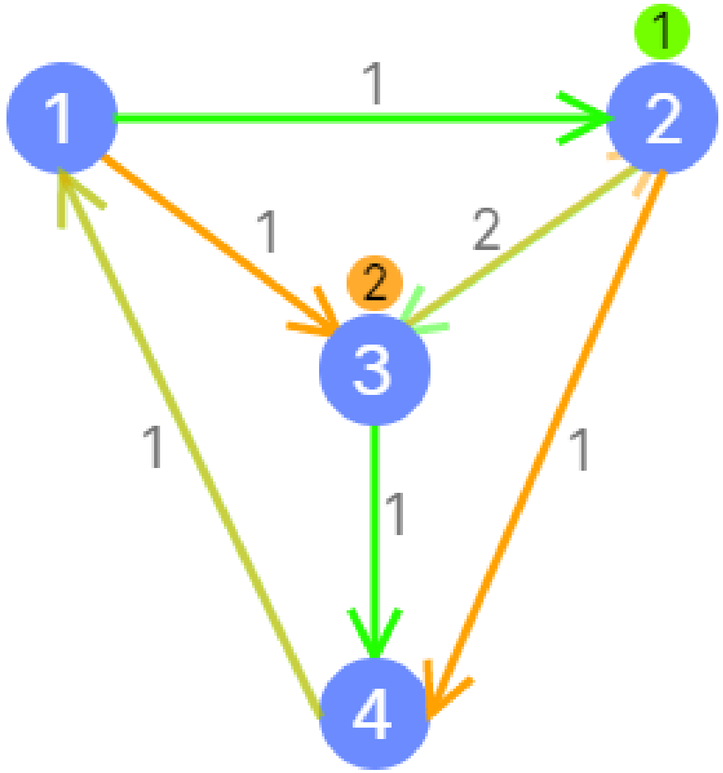


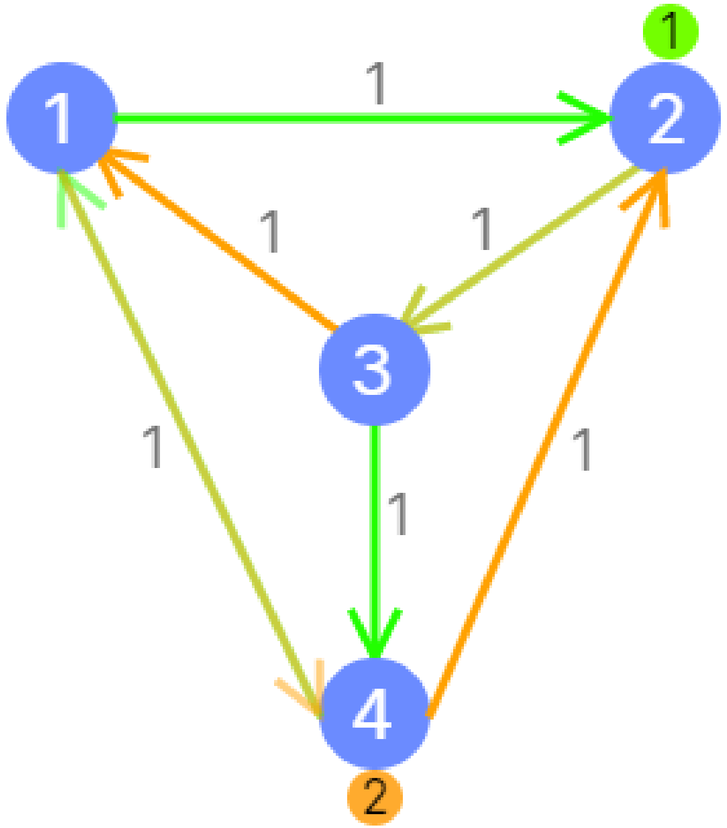
IV/ Résultats et conclusion



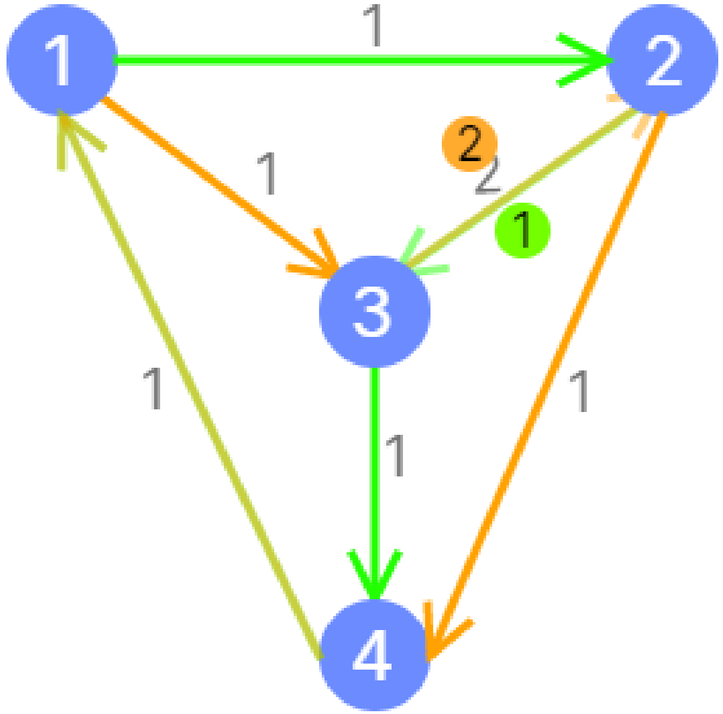


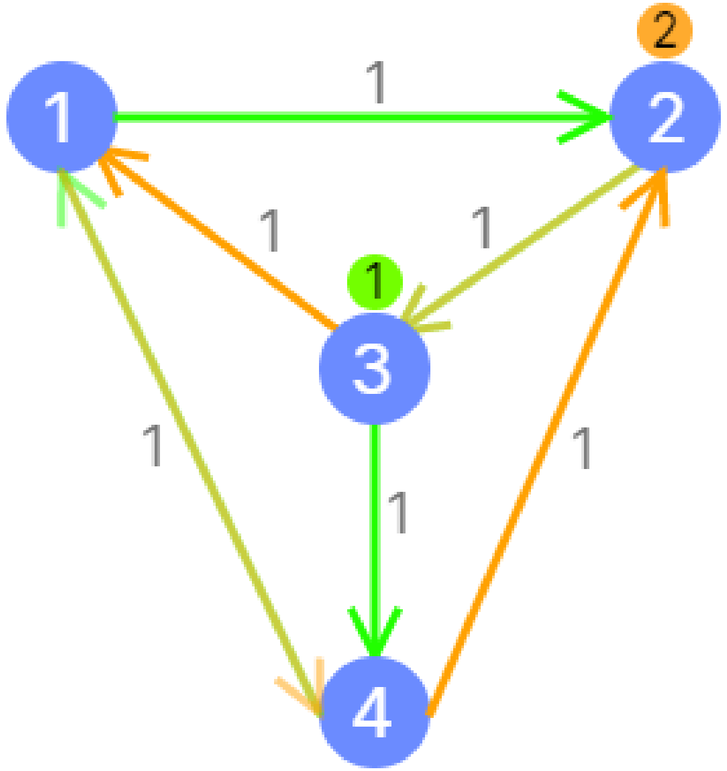
IV/ Résultats et conclusion



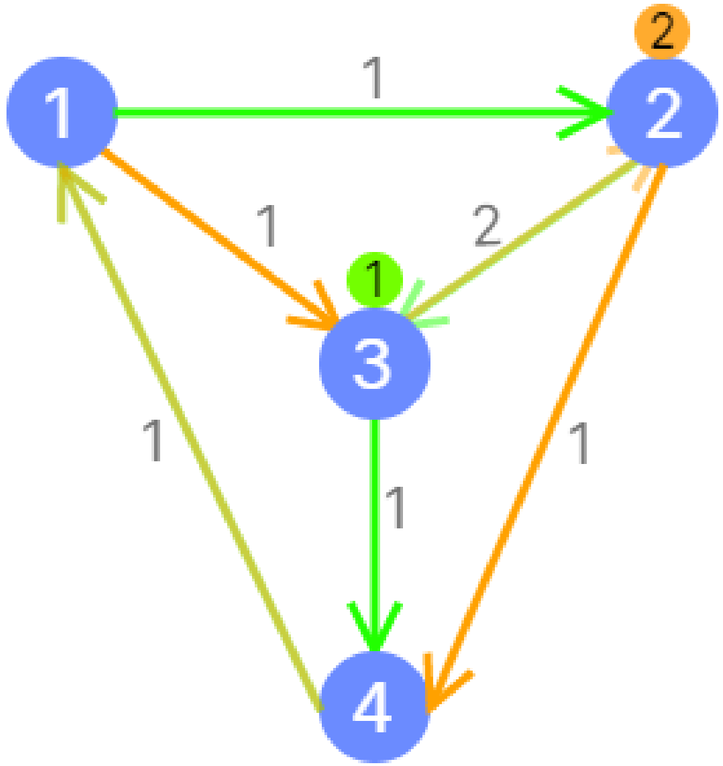


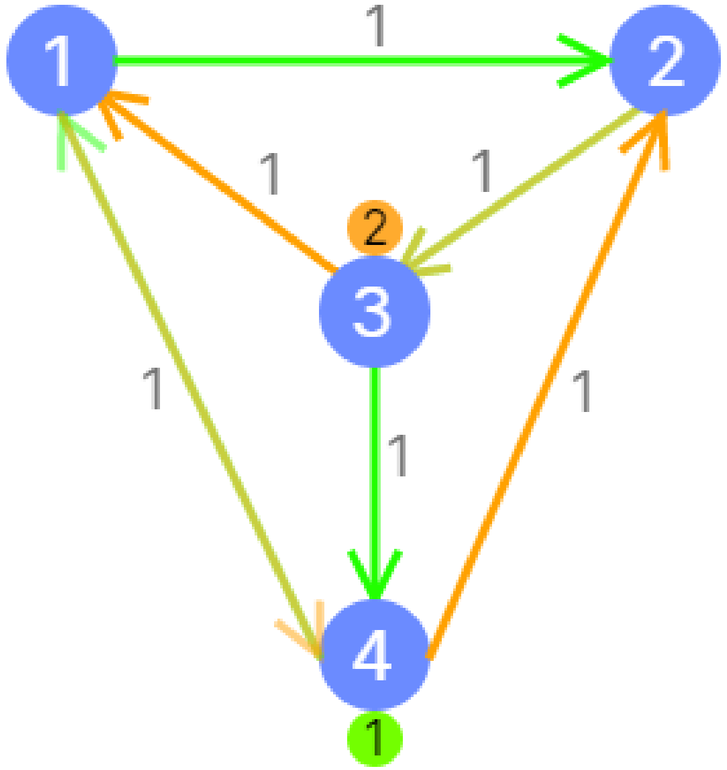
IV/ Résultats et conclusion



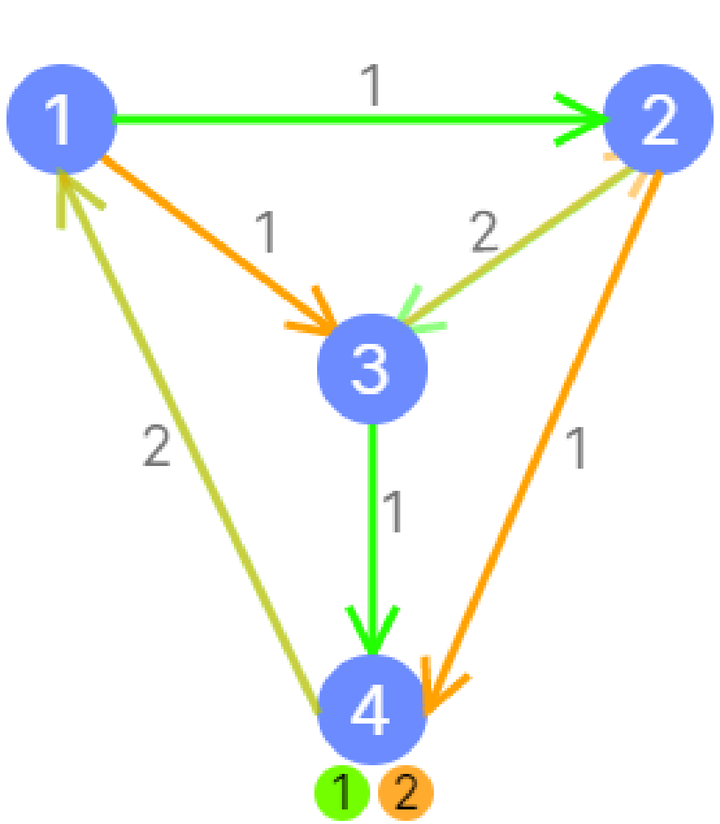


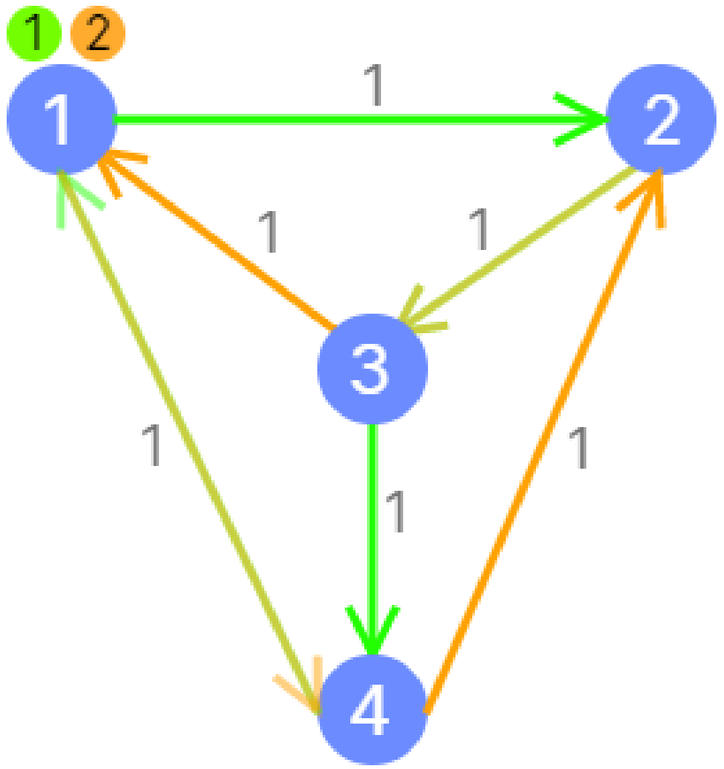
IV/ Résultats et conclusion



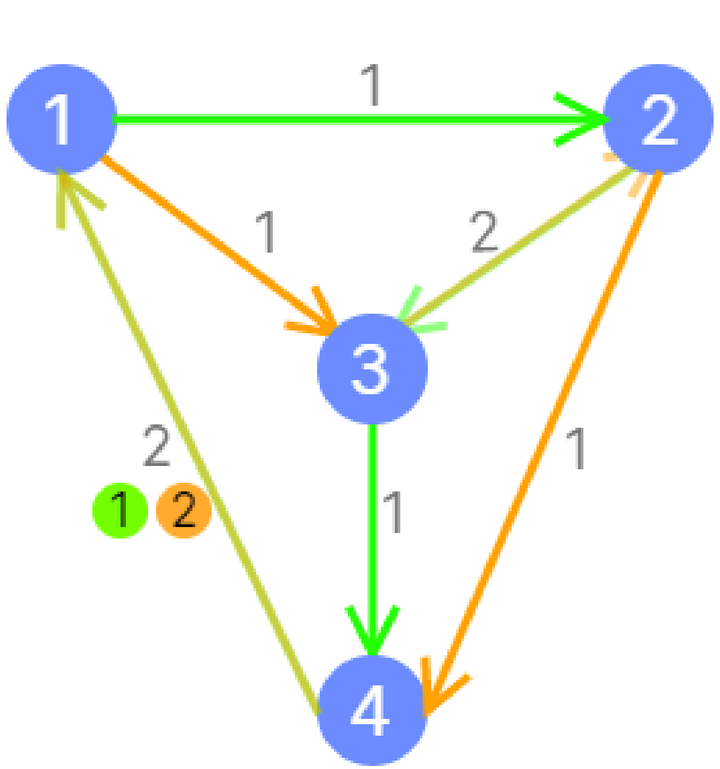


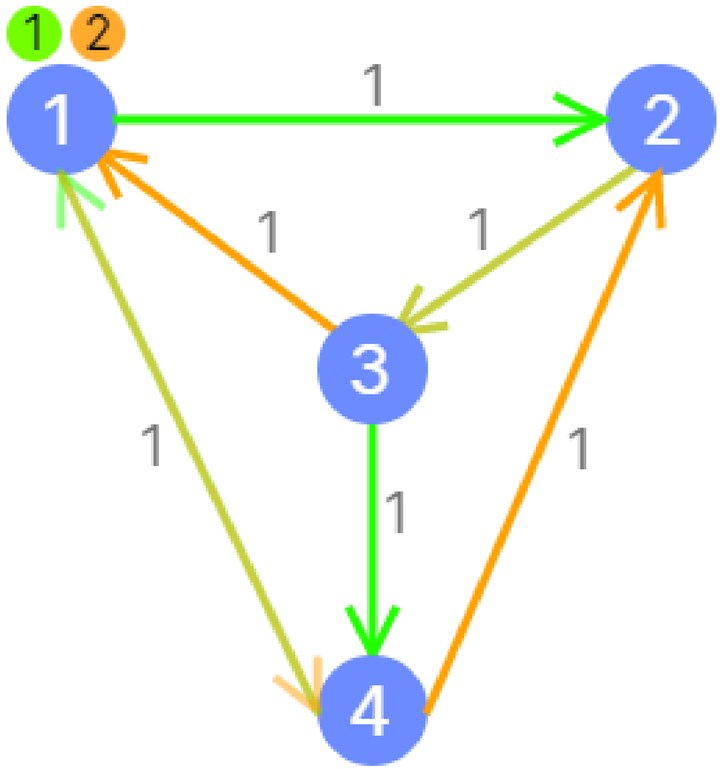
IV/ Résultats et conclusion



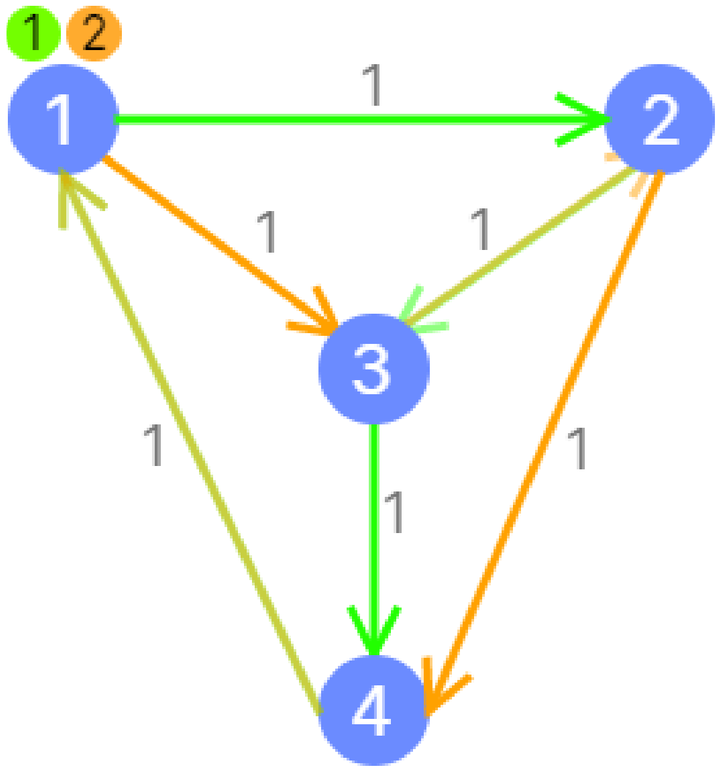


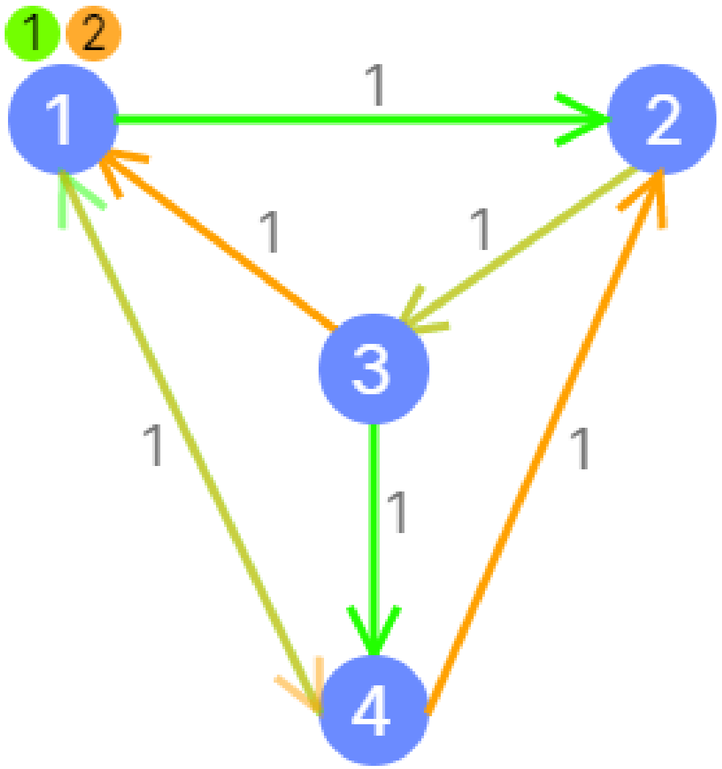
IV/ Résultats et conclusion





IV/ Résultats et conclusion





IV/ Résultats et conclusion

Soit G un graphe à n sommet et k colis

$$N = \frac{(n-1)!}{2}$$ (Nombre de cycle hamiltonien possible)

Nombre de situation à simuler :

$$\overbrace{\qquad\qquad k+N-1\qquad\qquad}$$



$$\binom{k+N-1}{N-1} = \frac{(k+N-1)!}{(N-1)!k!}$$

Complexité importante

IV/ Résultats et conclusion

Conclusion :

- intéressant pour gain de temps / efficacité
- modélisation différentes du poids des arrêtes
- distinctions entre les colis

Annexe

Sources:

[1] : https://algorithms.discrete.ma.tum.de/graph-games/tsp-game/index_en.html:

[2] : https://www.toncarton.com/produits/optimisateur-tournee/

[3] : https://www.mecalux.fr/blog/fonctionnement-robotisation-logistique-entrepot

[4] : https://storymd.com/journal/pwvvodvt7w-genetic-testing/page/4o8veo07k87-what-are-whole-exome-sequencing-and-whole-genome-sequencing

Annexe

Force Brute :

```
def force_brute(M, d):  1 usage
    n = len(M)
    R = [i for i in range(n) if i != d]
    # Fonction pour générer toutes les permutations de la liste L
    def perm(L):
        if len(L) <= 1:
            return [L]
        res = []
        for i in range(len(L)):
            p = L[i]
            reste = L[:i] + L[i + 1:]
            for pr in perm(reste):
                res.append([p] + pr)
        return res
    cycles = [[d] + p + [d] for p in perm(R)]
    best_c = float('inf')
    best_p = None
```

```
    for c in cycles:
        cout = 0
        for i in range(n):
            u = c[i]
            v = c[i + 1]
            cout += M[u][v]
        if cout < best_c:
            best_c = cout
            best_p = c

    return best_p, best_c
```

Annexe

Glouton :

```
def glouton(M, d):
    n,c,cout = len(M) , [d],0
    while len(c) < n:
        u = c[-1]
        suivant = None
        min_p = float('inf')
        for v in range(n):
            if v not in c and M[u][v] < min_p:
                min_p = M[u][v]
                suivant = v

        c.append(suivant)
        cout += min_p

    cout += M[c[-1]][d]
    c.append(d)
    return c, cout
```

Annexe

Held-Karp :

```python
def chemin_opti(sommets_restants, sommet_actuel):
    if len(sommets_restants) == 0:
        return (M[sommet_actuel][d], [d])

    etat = (sommets_restants, sommet_actuel)
    if etat in memoire:
        return memoire[etat]

    cout_min = float('inf')
    meilleur_chemin = []
    for prochain_sommet in sommets_restants:
        nouveaux_restants = tuple(s for s in sommets_restants if s != prochain_sommet)
        cout_suite, chemin_suite = chemin_opti(nouveaux_restants, prochain_sommet)
        cout_total = M[sommet_actuel][prochain_sommet] + cout_suite
        if cout_total < cout_min:
            cout_min = cout_total
            meilleur_chemin = [prochain_sommet] + chemin_suite
    memoire[etat] = (cout_min, meilleur_chemin)
    return memoire[etat]
```

Annexe

Held-Karp :

```
def held_karp(M, d=0):  1 usage
    nb_sommets = len(M)
    memoire = {}

    def chemin_opti(sommets_restants, sommet_actuel):...

    sommets = tuple(s for s in range(nb_sommets) if s != d)
    cout_final, chemin_final = chemin_opti(sommets, d)
    cycle_complet = [d] + chemin_final

    return cout_final, cycle_complet
```

Annexe

Génération de graphe aléatoire

```python
def graphe(n, m):  2 usages
    matrice = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            poids = random.randint( a: 1, m)
            poids = poids if random.random() <= 0.6 else float("inf")
            matrice[i][j] = poids
            matrice[j][i] = poids
    return matrice
```

Annexe

Simulation :

```
groupes = [ [ [i for i in range(nb)], cycle, 1] ]
```

```
def simulation(groupes,G):  6 usages
    #initialisation
    resultat = [0 for _ in range(len(groupes))]
    D_init = {}
    arret_max,encomb_max = (0,1),1
    for i in range(len(groupes)):
        cycle = groupes[i][1]
        arrete = cycle[0]
        if arrete in D_init:
            D_init[arrete].append(i)
        else:
            D_init[arrete] = [i]
    T = {0:D_init}
    #ecoulement du temps
    t = 0
    while 0 in resultat:
        D = T.get(t,  default: {})
        for arrete in D:
            lst = D[arrete]
            k = sum([len(groupes[i][0]) for i in lst]) - 1
            if k >= encomb_max :
                encomb_max = G[arrete[0]][arrete[1]] + k
                arret_max = arrete
            tps_add = G[arrete[0]][arrete[1]] + k
            for g in lst:
                avance = groupes[g][-1]
                if avance == len(G)+1:
                    resultat[g] = t
                    groupes[g][-1] = 1
                else:
                    arr_suiv = groupes[g][1][avance]
                    t_future = t + tps_add
                    if t_future not in T:
                        T[t_future] = {}
                    if arr_suiv in T[t_future]:
                        T[t_future][arr_suiv].append(g)
                    else:
                        T[t_future][arr_suiv] = [g]
                    groupes[g][-1] += 1
        t += 1
    return resultat,arret_max,encomb_max
```

Annexe

Algorithme

```
def methode1(groupes,G):  1 usage
    arretes_bannie = []
    div = 2
    t_deb,t_fin = float('inf'), 0
    meilleur_tps = t_deb
    while t_fin < t_deb:
        rslt_deb,art_mx,pd_mx = simulation(groupes, G)
        t_deb = max(rslt_deb)
        arretes_bannie.append((art_mx,pd_mx))
        #modif graphe
        M = [l.copy() for l in G]
        for ar,pd in arretes_bannie:
            M[ar[0]][ar[1]] = pd
        #nouveau cycle
        _,cycle_opti = held_karp(M, d: 0)
        #essai
        grp_tmp = []
        sous_grp = repartition([i for i in range(nb)],div)[::-1]
```

Annexe

Algorithme

```
            for k in range(len(sous_grp)-1):
                grp_tmp.append([sous_grp[k],groupes[k][1],1])
            grp_tmp.append([sous_grp[-1], cycle_opti, 1])
            rslt_fin,_,_ = simulation(grp_tmp, G)
            t_fin = max(rslt_fin)
            groupes = grp_tmp
            div += 1
        else:
            break
        if t_fin <= meilleur_tps:
            meilleur_tps = t_fin

return meilleur_tps
```